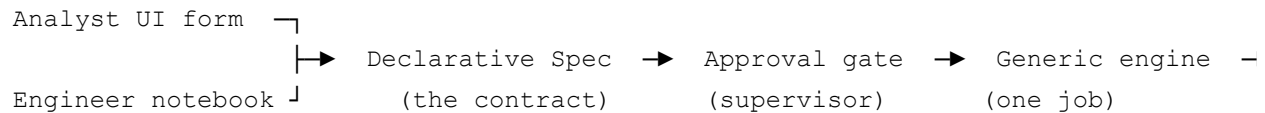


ARW Feature Store — POC

A runnable proof-of-concept of a **governed, spec-driven feature store** for fraud velocity features, with **two front doors** (an analyst UI and an engineer notebook) feeding one approval-gated pipeline. Built on a sample **credit-card authorization** dataset.

It demonstrates the exact design we sketched, in miniature:



Production mapping: **pandas** → **Spark**, **CSV** → **Iceberg on Hadoop**, **SQLite** → **Postgres**.
The architecture and the spec are identical; only the backends swap.

What's inside

Path	What it is
<code>data/generate_data.py</code>	Generates a synthetic 29K-row credit-card auth table
<code>feature_store/spec.py</code>	<code>FeatureSpec</code> — the declarative contract both doors produce
<code>feature_store/engine.py</code>	The one generic engine : spec → point-in-time-correct values
<code>feature_store/registry.py</code>	SQLite registry + <code>submit</code> → <code>approve</code> governance workflow + audit
<code>feature_store/store.py</code>	Production-gated materialization + point-in-time training-set join
<code>feature_store/tiles.py</code>	Scalable data layer — scan source once into daily tiles
<code>feature_store/rollup.py</code>	Cheap window features from tiles (no source scan)

specs/*.yaml	Example feature specs
notebook/feature_store_poc.ipynb	Engineer door — full walkthrough end-to-end
notebook/scalable_tiling_poc.ipynb	Data approach — tile-then-rollup, scan-once proof
app/streamlit_app.py	Analyst door — no-code feature-builder UI + approval queue

Run it

```
# 1. install
pip install -r requirements.txt

# 2. generate the sample data
python data/generate_data.py

# 3a. engineer path — open the notebook
jupyter lab notebook/feature_store_poc.ipynb

# 3b. analyst path — launch the UI
streamlit run app/streamlit_app.py
```

The Streamlit app has three tabs: **Build a feature** (form → spec → preview → submit), **Approval queue** (supervisor approves → auto-materializes), and **Catalog & audit**.

The four sample velocity features

Feature	Definition
denials_30d	count of <code>DENY</code> auths per account, trailing 30 days
txn_count_7d	transaction count per account, trailing 7 days
amount_sum_30d	total spend per account, trailing 30 days
distinct_mcc_7d	distinct merchant categories, trailing 7 days

You can build any combination of `count` / `sum` / `avg` / `min` / `max` / `count_distinct` over

any field, window, and filter — from the UI or the notebook.

What the POC proves

- 1. **One declarative spec drives everything** — no per-feature Spark/pandas code.
 - 2. **Two doors, one path** — analyst form and engineer notebook produce the *same* spec and go through the *same* governance.
 - 3. **The governance gate is real** — nothing materializes without supervisor approval; every action is in the audit log.
 - 4. **Point-in-time correctness** — features aggregate only events *strictly before* each row (no label leakage). The first event per account is always 0, which proves it.
 - 5. **The 30 → 90 day change is a spec edit, not a code change** — the headline requirement.
-

How this maps to production on your stack

POC	Production
<code>pandas rolling</code>	Spark windowed aggregation job (one generic job)
CSV in <code>feature_tables/</code>	Iceberg tables on HDFS (the Hadoop cluster)
SQLite <code>registry.db</code>	Postgres registry
Streamlit form	the analyst feature-builder UI
<code>approve_feature()</code>	git PR approval / supervisor sign-off in the UI
<code>build_training_set (merge_asof)</code>	Trino point-in-time join over Iceberg

The data approach (for huge Parquet auth tables)

Recomputing each window feature from raw records is a near-full scan **per feature, per run** — too expensive at billions of rows. The fix (`tiles.py` + `rollup.py` , demonstrated in `notebook/scalable_tiling_poc.ipynb`):

Scan the source **ONCE** into **daily per-entity tiles**; every window feature is a cheap roll-up over tiles.

```
raw auths --(one scan)--> account_daily_tiles --(rollup)--> denials_30d
                                                    --(rollup)--> denials_90d    (same t
                                                    --(rollup)--> txn_count_7d
```

	Naive	Tile + roll-up
Source scans for N features	N	1
30 → 90 change	full recompute	roll-up over existing tiles
Steady-state daily cost	re-scan history	one day of new data (incremental)
Reads at feature time	huge raw table	tiny daily tile table

The POC instruments a **source-scan counter** to prove the raw table is touched exactly once across all features and the 30→90 change.

Production notes: build tiles with a daily Spark `GROUP BY` over one **date partition**, appended to an **Iceberg** tile table; keep the **massive source auths as Parquet**, write the **small tiles + features as Iceberg** (incremental reads + time-travel); **partition source by date** and project only needed columns so the one scan is bounded. Distinct counts use daily set-union (low cardinality) or mergeable **HyperLogLog** sketches (high cardinality).